

File I/O and CSV Module

This document contains useful information on how to read, write, and parse files.

Note: The terminology included in this document and a basic understanding of file I/O is course content and therefore may be included on exams or quizzes.

Table of Contents

Introduction	2
Terminology	3
File Input/Output (File I/O)	3
Access Modes	3
open()	4
close()	4
file.read()	4
file.readline()	4
file.readlines()	4
seek()	4
Reading CSV Files	5
csv.reader	5
csv.DictReader	7
Optional Parameters	8
Writing CSV Files	9
csv.writer	9
csv.DictWriter	10
References	11

Introduction

CSV files

Comma-separated-value files, or CSV files, are text files that store information in an easy-to-read, tabular format. Every piece of data in these files are distinguishable from each other because they are separated by commas, or some other delimiter (despite its name, CSV files don't necessarily have to use commas to separate each piece of data). Each row represents some sort of individual record of the data, and each column will represent an attribute of each record. For example, suppose we want to go to the beach for spring break, and we've created a CSV file that holds information about different vacation homes (see image below). Each row would hold information regarding a different house, and each column would hold information about particular attributes for each of those houses, such as the number of bathrooms, the number of bedrooms, or the location of the house.

beachVacation.csv

```
Location,Bathrooms,Bedrooms,Pool,Rating,CostPerNight
Malibu,4,3,True,4.0,750
Miami,3,3,True,4.3,680
Oahu,1,4,False,3.4,400
Cape Cod,4,4,True,4.8,1450
Outer Banks,3,3,False,4.4,650
```

CSVs in other forms

The tabular formatting of text in a CSV file might remind you of a spreadsheet. CSV files are versatile in that they can be rendered and manipulated in any spreadsheet program, including Microsoft Excel. However, As mentioned before, CSV files are simply text files. To be more specific, CSV is a special format that text files can be in. Thus any file with the extension .txt (or any text file) can be considered a CSV file **as long as** it follows the CSV format.

Python's csv module

Python has a built-in module specifically for handling and interpreting the data in CSV files. The file must first be opened as a file object. Then with the csv module, we can either transfer the text from a CSV into a Python data structure or we can take a Python data structure and insert its contents into a CSV file. In order to use the csv module, we must import it! So if you plan to interact with CSV files using the standard Python library then you want to **import csv**!

Terminology

CSV file - stands for Comma-Separated Value file; a text file that holds data that is stored in a tabular format, a common format for importing and exporting spreadsheets and databases

TSV file - stands for Tab-Separated Value file; a text file that holds data that is stored in a tabular format where each feature is delimited by a tab

Delimiter - the character (or series of characters) that separates each feature throughout the file. By default, the delimiter in CSV files is a comma

Encoding - a parameter that may need to be specified when opening a CSV file as a file object. In short, it is the way that a computer stores characters as bits. For this class, most of the time you will either need to include `encoding = "utf8"` or `encoding = "iso-8859-1"` when opening a file.

File Object - an object in Python that we can create to read, edit, and manipulate data from a file

Parse - analyze and subdivide text (in this case a CSV file) into logical syntactic components

Cursor - tells the read function (and many other I/O functions) where to start from. To set where the cursor is, you use the `seek()` function (*found below*).

File Input/Output (File I/O)

Python has a built-in file input/output module that allows us to extract data from txt or CSV files. File I/O works similarly to the CSV module - we will highlight the differences later in this document.

Access Modes

To determine what action you want to take with a file, Python has access modes. These are passed into the parameter of the `open()` method.

Access Mode	Definition	Letters Denoted
Read	Reads the text from file	"r"
Write	Writes text to a new file with the name specified OR overwrites an existing file	"w"
Append	Adds text to the end of an existing file	"a"

If you don't specify an access mode as a parameter, Python will assume you are reading text ("r").

`open()`

Open any text file using the `open()` method. There are two ways to use it:

1. `fin = open("file_name.txt", "r")`
`text = fin.read()`
`fin.close()`
2. `with open("file_name.txt", "r") as fin:`
`text = fin.read()`

The difference is that the first way requires you to close the file deliberately after reading the text, while the second way will close the file automatically. We use the second way as convention.

`file.close()`

Files **must** be closed in order to keep any changes made. You only need to use this method when you are using the first method to open the file.

`file.read()`

This method returns one long continuous string of the entire file, including all whitespace. We typically use `strip()` with File I/O methods to remove the whitespace. Include an integer parameter to specify a certain string length.

`file.readline()`

This method returns a string of one line at a time, including all whitespace. If you only call it once, it will return one line. If you call it again, it will return the next line.

`file.readlines()`

This method returns a list of every line as a string, including all whitespace.

`read()`, `readline()`, and `readlines()` all depend on the cursor object, which dictates where you are in your file. If you call `read()` or `readlines()`, you are reading the entire file so if you try to call `read()` or `readlines()` again right after, you will get an empty string or list because your cursor is at the very bottom of the document. `readline()` will return one line at a time, moving the cursor by one every time you call it.

`seek()`

The Python file method `seek()` sets the file's current position at a certain position. We use a cursor object to keep track of our location. The `seek` method utilizes the following syntax.

```
fileObject.seek(offset[, whence])
```

The fileObject will be the name of any object that you assign the open variable to. The offset variable is required and denotes the position of the read/write pointer within the file that you are “seeking”. The whence argument is optional and defaults to 0, which means absolute file positioning, other values are 1 which means seek relative to the current position and 2 means seek relative to the file's end. Below are a few examples with different optional arguments.

```
openfile.seek(45,0)
```

The above line would move the cursor to 45 bytes/letters after the beginning of the file.

```
openfile.seek(10,1)
```

This above line would move the cursor to 10 bytes/letters after the current cursor position.

```
openfile.seek(-77,2)
```

This above line would move the cursor to 77 bytes/letters before the end of the file (notice the negative sign before the 77)

Invisible Characters in Files

Everything from a file is read into Python as a string (str). Frequently these strings will need to be converted to numeric values. It is important to be very aware of exactly what characters the data you have read contains. For example, at the end of every visible line in a file is an invisible character that represents the newline character. Escape sequences are used to represent invisible characters like tabs and newlines. The sequence “\n” represents a newline. The string “\t” represents a tab character. Other escape sequences you will need for this class are “\” to represent a single quote, “\” to represent a double quote, and “\” to represent the backslash itself.

Reading CSV Files

In the previous sections, we learned how to read text files into Python one line at a time, or even all at once in one string that encompasses all the contents of the file. `.readline()` and `.readlines()` can be tedious to work with, especially when we have newline characters at the end of every line. This is where the CSV module comes in: since the CSV format has its own special pattern, rather than using `.readline()` or `.readlines()` to read in each row of data as its own string and then split each string into its individual delimiter, it is more efficient to use the csv module, as it significantly increases the efficiency of our code. Given a file in CSV format, we can parse the text into its individual pieces of data, saving and grouping them into different data structures without having to worry about doing the parsing ourselves. We can read CSV files using two different functions from the csv module, namely, reader and DictReader. It is important to note that when we read from the CSV file, **all data will be read in as STRINGS**. You will need to cast the data to the appropriate types if you wish to manipulate them later.

csv.reader

With CSV reader, we can translate a CSV file into a **list of lists**, where each list will represent a row of data. Using the vacation homes example from above, we want the csv reader to return something that looks like the following image:

```
[['Location', 'Bathrooms', 'Bedrooms', 'Pool', 'Rating', 'CostPerNight'],  
 ['Malibu', '4', '3', 'True', '4.0', '750'],  
 ['Miami', '3', '3', 'True', '4.3', '680'],  
 ['Oahu', '1', '4', 'False', '3.4', '400'],  
 ['Cape Cod', '4', '4', 'True', '4.8', '1450'],  
 ['Outer Banks', '3', '3', 'False', '4.4', '650']]
```

This is useful in that each row will represent a particular item (in this case, a house), and the order of items in a list will stay the same, thus we can index the inner list to get particular values. For example, to get the number of bathrooms we would take the 1st index of each list, and to know if the house has a pool or not, we look at the 3rd index.

Application

The code to translate a CSV file into this list of lists is very simple:

```
1 with open("csvFileName.csv", "r") as fin:  
     2 reader = csv.reader(fin)  
     3 readerList = [line for line in reader]
```

The general outline for reading a csv file using the `csv.reader` function is as follows,

1. Open the file for reading as a file object (you can use the context manager or create a file object, then close the file later)
2. Create a reader object using the `csv.reader()` function
 - Don't forget to import `csv`!
 - Note that the `csv.reader()` function returns a **csv reader object**. It is iterable (you can use it in a for each loop), but NOT subscriptable (you can't index into it)
 - See section "Optional Parameters"
3. Cast the reader object to be a list
 - Since we cannot index into the reader object it is necessary to create a list of lists representing the contents of the csv file
 - i. This can be accomplished by either casting the reader object to a list (e.g. `readerList = list(reader)`)
 - If there are particular columns or rows that contain data you don't need, you may eliminate the extra lines by:
 - i. selectively indexing the areas of importance (e.g. `readerList = [i[0] for i in reader]` will only retrieve the first column)

- ii. adding an if statement to your list comprehension when you iterate through the reader object to check a condition
- iii. use the next function to skip the header row (e.g. `headers = next(reader)`) or slice `readerList` to only include the rows you need (e.g. `readerList[1:]` will eliminate the header line).
 - 1. You can also use `.insert(index, name)` to add values into the middle of a row

csv.DictReader

```
[{'Bathrooms': '4',
  'Bedrooms': '3',
  'CostPerNight': '750',
  'Location': 'Malibu',
  'Pool': 'True',
  'Rating': '4.0'},
 {'Bathrooms': '3',
  'Bedrooms': '3',
  'CostPerNight': '680',
  'Location': 'Miami',
  'Pool': 'True',
  'Rating': '4.3'},
 {'Bathrooms': '1',
  'Bedrooms': '4',
  'CostPerNight': '400',
  'Location': 'Oahu',
  'Pool': 'False',
  'Rating': '3.4'}
 ...]
```

As mentioned above, there are times when the data we are given starts with a header line that may not seem important to include in our data structures. However, the `csv.DictReader()` constructor puts the header line to good use, using them as keys for dictionaries that represent each item.

With the help of the `csv` module's `DictReader` constructor, we can transform the data in a CSV file into a **list of dictionaries**, where each dictionary represents a different item/entry.

If the CSV file looks like this:

```
header1, header2, header3, header4...
data11, data12, data13, data14...
data21, data22, data23, data24...
```

Then parsing it using `csv.DictReader()` will transform it into this...

```
[{header1: data11, header2: data12, header3: data13, header4: data14 ... },
 {header1: data21, header2: data22, header3: data23, header4: data24 ... }...]
```

This time, if we want to access each house's number of bathrooms, we will need to index each inner dictionary using the `'Bathrooms'` key..

Application

The application of the `csv.DictReader()` is very similar to that of `csv.reader` with some slight adjustments:

```
1 with open("csvFileName.csv", "r") as fin:
2     dictReader = csv.DictReader(fin)
3     listOfDicts = [dict(line) for line in dictReader]
```

The general outline for reading a csv file using the `csv.DictReader` constructor is as follows,

1. Open the file for reading as a file object (same as reader)
2. Create a DictReader object using `csv.DictReader()` constructor
 - The D and first R MUST be capitalized
 - Don't forget to `import csv`!
 - Note that the `csv.DictReader()` constructor creates a **csv DictReader Object**. It is iterable, but NOT subscriptable (you can't index into it)
3. Convert the DictReader object to a list of dictionaries
 - In Python 3.7, if we do not cast each line to be a dictionary, each line will by default be an `OrderedDict` object, which does not behave the same as a Python dict type. Therefore, we will have to cast each line to a dictionary using Python's built-in `dict()` constructor for further use and manipulation.

Optional Parameters

When creating a csv reader or DictReader, there are optional parameters that we can specify to better suit the data in the CSV file we are dealing with. These are just a couple of common ones you might have to deal with

```
csv.DictReader(  
    fin,  
    fieldnames = ("Location", "Bathrooms", ...),  
    quotechar = "'",  
    delimiter = ";"  
)
```

Fieldnames

Used only in `DictReader()`

Suppose you are trying to create a DictReader. In the event that your CSV has **no header line**, and therefore you have no header values to act as the keys of your dictionaries, you can define your own inside the `fieldnames` parameter, which can take in a sequence of your desired header values in the order that they appear in your CSV. If the `fieldnames` parameter is not specified, the values in the first row of the file will be used as the header values.

Delimiter

Used in both `reader()` and `DictReader()`

As mentioned earlier in this unit, the default delimiter when reading a CSV is the comma character. However, not all CSV files use commas to separate their data. Some might use a semicolon, or even a number or a letter. In these cases, you can specify exactly what the delimiter of the CSV file you're reading is in the `delimiter` parameter.

Quotechar

Used in both `reader()` and `DictReader()`

As an example, let's say the delimiter of your CSV file is a space. In the CSV file, you have a name attribute, and for one person's name happens to have a space in it, such as Hannah Ann. If you leave the name as it is, the name might be mistranslated to be "Hannah" rather than Hannah Ann. To avoid this misalignment of data, we can surround her name in double quotes: "Hannah Ann". In this case, the " is our quotechar, and it happens to be the default quotechar for CSV's.

Writing CSV Files

While reading is being given a CSV file and translating it to Python Code, we can do the opposite with writing, where we have Python code that we wish to write to a CSV file. There are two items from the CSV module that we use to do this: the `csv.writer()` function and the `csv.DictWriter()` class. Both are excellent ways of writing to a CSV, however one function might make more sense to use over the other depending on the type of data structure that currently holds the data to be written. `writer()` and `DictWriter()` are pretty intuitive in that their functions are the exact opposite of their reader counterparts, but unlike the reader and `DictReader` where the data is automatically translated as strings, the initial data you wish to write to a CSV file can be of any data type.

csv.writer

`csv.writer()` is more useful if your data is currently stored in a list or a list of lists. The first two steps of using a csv writer object consist of creating a file object for writing, and then creating a writer object using `csv.writer()`.

```
with open("csvFile.csv", "w", newline = "") as fout:
    writer = csv.writer(fout)
```

We add the `newline` parameter when creating the file we are writing to to indicate that each row written to this file should be one after the other, with no gap or empty line between them. Following the creation of our writer object, we can write lines to the CSV in one of two ways:

Way #1: `writerow()` takes in a list of the data you would like to write (order of the data matters, and should be consistent throughout the file to maintain the tabular nature of a CSV) and writes one new row of data in your outfile. You will need to repeat this line of code for as many rows of data you wish to write. You could also include this line in a for loop to reduce lines of code.

```
writer.writerow([data1, data2, data3])
```

Way #2: `writerows()` takes in a list of lists, where each inner list will be a new row/entry in the CSV, and writes multiple lines of data in your outfile. Once again, order of the data matters, and should be consistent throughout the file.

```
writer.writerows([[data11, data12, data13],
                  [data21, data22, data22]])
```

csv.DictWriter

`csv.DictWriter()` is more useful if your data is currently stored in a dictionary or a list of dictionaries. Like with the previous functions in this section, we must first create a file object, this time for writing, and then we can create our DictWriter object using `csv.DictWriter` (again note the D and the W *must* be capitalized)

```
with open("csvFile.csv", "w") as fout:
    dw = csv.DictWriter(fout, fieldnames = ['key1', 'key2', ...])
    dw.writeheader()
```

When creating the DictWriter object, **fieldnames** is a *required parameter* that we must specify. As mentioned in the DictReader section, the keys in the list of dictionaries correspond to the headers of the CSV file. The fieldnames parameter will be exactly that: the header line of the CSV file you're writing to. For this parameter you can pass in a list of hardcoded values, or if your headers can already be found in the keys of a dictionary of your data, you can use the `aDict.keys()`. Once we have the fieldnames defined, writing the header line in the CSV file is really easy:

```
dw.writeheader()
```

We can also use `.writerow()` and `.writerows()` for a DictWriter object, however the usage is slightly adjusted. For the writer, the `.writerow()` function took in a list, however for the DictWriter, it will take in a dictionary, where each dictionary represents one item/entry, the keys are the headings, and the values are the corresponding data. Similarly, for the DictWriter, `.writerows()` will take in a list of dictionaries.

```
dw.writerow({'key1': 'value1', 'key2': 'value2', ... ,})
dw.writerows([{"key1": "value11", "key2": "value12"}, {"key1": "...",
...}])
```

References

<https://docs.python.org/3.9/library/csv.html>

<http://kunststube.net/encoding/> - for “encoding” definition